

또, 드릴게요! 이기적이 준비한 선물



또기적

기출분석자료집

프로그래밍기능사 필기

✓ Python 핵심이론

✓ 2026년 제1회 필기 기출복원문제



Python

학습 방향

파이썬의 기초 문법과 문자열/리스트 슬라이스와 제어문, 반복문, 함수에서 다수 출제되고 있습니다. 교재의 내용은 기본적인 내용 위주로 소스를 구성하였으며, 기출문제를 바탕으로 작업했습니다. 기본 소스를 분석하고 결과를 적을 수 있도록 학습하세요.

난이도

- 하 SECTION 01 Python 언어 기본, 자료형과 연산자
- 중 SECTION 02 Python 문자열 슬라이스와 군집 자료형
- 중 SECTION 03 Python 제어문과 반복문
- 상 SECTION 04 함수, try, match

Python 언어 기본, 자료형과 연산자

빈출태그 파이썬 • 주석 • print • input • 자료형 • 연산자

01 Python 기본

1) 파이썬의 특징

- 1989년 네덜란드 출신의 귀도 반 로섬(Guido van Rossum)에 의해 개발된 스크립트형 프로그래밍 언어로써 문법이 간결하여 쉽게 작성할 수 있고 결과를 빠르게 확인할 수 있다.
- 확장 가능한 다양한 라이브러리가 존재하여 생산성이 높아 개발 속도가 빠르다.

2) 파이썬 대화형 셸과 모듈

① 대화형 셸에서 명령어 입력

- 대화형 셸에서 아래와 같이 입력한다.

```
>>> 5+3
8
>>> a=5
>>> b=4
>>> a+b
9
>>> print('hello handaman')
hello handaman
```

+ 더 알기 TIP

- 파이썬은 인터랙티브 셸(대화형 셸)에서 명령어를 입력하면 바로 결과가 나옵니다.
- 파이썬 명령 프롬프트(>>>)는 파이썬 명령어를 실행하는 기본 실행 환경입니다.
- 문자열 구분은 작은 따옴표(')와 큰 따옴표(") 모두 가능합니다.

② 모듈 형태로 Python 명령어를 저장

- 텍스트 에디터에서 파이썬 코드를 작성하여 확장자가 .py인 하나의 소스파일로 생성 후 실행한다.

```
print('한')
print('다')
print('맨')
```

결과	한
	다
	맨

3) 파이썬 주석

- 주석이란 프로그램 실행과 관계없는 문장을 말한다.

① 한 줄 주석 달기

- 한 줄만 주석 처리를 하려면 주석 왼쪽에 '#'을 입력하고 작성한다.

```
print('안녕하세요')      #hello
print('한다맨')          #handaman
```

결과	안녕하세요
	한다맨

- 'hello'와 'handaman'은 실행되지 않는다.

② 여러 줄 주석 달기

- 주석 문장을 작은 따옴표 ' 또는 큰 따옴표 " 세 번으로 묶어준다.
- 형식 : ''' 설명1..설명2.. ''' 또는 """ 설명1..설명2.. """
- 예제

```
print('A')
...
여러 줄의 주석을 처리하려면 작은 따옴표 또는 큰 따옴표 세 번으로 묶어주세요.
print 함수는 출력한 후, 자동으로 줄 바꿈을 합니다.
...
print('B')
```

결과	A
	B

4) 파이썬의 블록

- 같은 블록 영역의 명령어들은 동일한 간격으로 들여쓰기를 해야 한다.
- 일반적으로 **Tab**으로 들여쓰기 간격을 만들거나 **Space Bar**를 이용하여 4칸 띄운다.

① 올바른 들여쓰기 예제

```
n=10                                # n에 10을 줍니다.
if n%2==0:                          # n과 2를 나눈 나머지가 0이니? 참입니다.
    print("even")                   # 'even'을 출력합니다.
    print("짝수")                   # '짝수'를 출력합니다.
else:
    print("odd")                    # 거짓인 경우 출력하는 문장
    print("홀수")
```

결과	even
	짝수

+ 더 알기 TIP

- n=10, if, else는 최상위 문장으로 가장 왼쪽에 위치하며, print문은 if와 else에 속한 문장으로 들여쓰기로 구분합니다.
- C언어와 Java는 명령문이 여러 줄인 경우 종괄호 '{ }'로 구분하지만 파이썬은 들여쓰기로 구분합니다. 단, 들여쓰기는 왼쪽 경계선을 정확하게 맞춰 코딩해야 합니다.

② 잘못된 들여쓰기 예제

```
print("even")
print("짝수")
```

+ 더 알기 TIP

위의 print문은 들여쓰기가 틀렸기 때문에 SyntaxError 메시지가 실행되며, unexpected indent 메시지가 출력됩니다.

5) 변수, print, input

① 변수 작성 규칙

- 변수는 '문자, _(언더바), 숫자'만 사용할 수 있다.
- 변수의 첫 글자로는 문자와 _(언더바)만 사용할 수 있다.
- 변수명은 대소문자를 구분한다.
- 변수명은 파이썬의 함수 이름이나 예약어를 사용할 수 없다.

② 변수와 print 사용 예제

• 예제1

<pre>a=10 # a라는 컵에 10을 넣는다. print(a) # a에 담겨있는 10을 출력한다. print("a") # 문자 'a'를 출력한다.</pre>	
결과	10 a

• 예제2 : 에러가 발생하는 경우

<pre>a=10 # a라는 컵에 10을 넣는다. print(a) # a에 담겨있는 10을 출력한다. print("a") # 문자 'a'를 출력한다. print(b) # b 변수에는 값이 없어서 에러가 발생한다.</pre>	
결과	10 a line 4, in <module> print(b) NameError: name 'b' is not defined

• 예제3

<pre>a=10 b=5 print(a, b) # 콤마(,)로 구분하면 한 줄로 출력된다. b=7 print(a+b) print(True) print(False) c = "프로그래밍" d = "기능사" print(c + d) print("프로그래밍" + "기능사")</pre>	
결과	10 5 17 True False 프로그래밍기능사 프로그래밍기능사

+ 더 알기 TIP

True, False : 참, 거짓으로 true와 false는 인식하지 않습니다.

③ 변수와 input 사용 예제

• 예제1

```
name = input('이름이 뭐예요 ? ') # 입력받은 문자열은 name에 담는다.  
print(name, '안녕')
```

결과

이름이 뭐예요 ? 한다맨
한다맨 안녕

+ 더 알기 TIP

```
name = input( ) # 메시지 없이 사용자의 입력값을 name에 저장한다.  
print(name, '안녕')
```

02 자료형

- 자료형이란 데이터의 종류를 의미한다.
- 변수를 만들 때 사용자가 자료형을 결정하지 않아도 파이썬 내부에서 자동으로 자료형을 판단하여 적용한다.
- 자료형 확인은 `type()` 함수로 알 수 있다.
- 필요에 따라 자료형을 변경할 수 있다.
- 각 자료형의 특징을 이해하면 효율적인 코드를 만들 수 있다.

1) 숫자형

- 연산이 가능하며 내장함수 사용이 가능하다. 예) `round()`, `range()`, `pow()` 등
- 정수형(int)과 실수형(float)이 있다.

2) 논리형

- 참(True)과 거짓(False)을 나타내는 자료형으로 주로 비교 또는 논리 연산자로만 들어진다.
- 조건문에 많이 활용된다.

3) 문자열형(str)

- 작은 따옴표(') 또는 큰 따옴표(")에 감싸져 있다.
- 연산이 불가능하다(예외 : 문자+문자, 문자*정수 또는 정수*문자).
- 문자열을 다루는 다양한 함수들이 존재한다. 예) `split()`, `upper()`, `lower()`, `replace()` 등
- 다른 언어와는 달리 문자와 문자열을 따로 구분하지 않는다.

• 예제

```
a=5
b='5'
c=5.0
print(a+a)
print()           # 줄바꿈
print(b+b)        # 문자와 문자를 결합
print(a*b)        # 문자 5를 5회 출력
print()
...

print(a+b)        # 정수+문자는 안 됨
print(b*c)        # 문자*실수도 안 됨
...

print('안녕하세요')
print("안녕하세요") # print("안녕하세요")처럼 '과 "의 혼용은 안 됨
```

결과	10
	55
	55555
	안녕하세요
	안녕하세요

03 자료형 변환

- 파이썬은 사용자가 자료형을 결정하지 않기 때문에 편리하지만, 사용자의 의도와 다른 자료형이 될 수도 있는 단점이 있다.
- 이러한 단점을 해결하기 위해 파이썬은 각 데이터들의 자료형을 원하는 자료형으로 쉽게 변경할 수 있다.

1) 자료형 변환(typecasting) 방법

- 원하는 자료형 함수에 값을 넣는다.
- 형식

```
str(10)           #숫자 10을 문자 10으로 변경
int('10')        #문자 10을 숫자 10으로 변경
int(12.5)        #실수 12.5를 정수 12로 변경
list('hello')     #문자열 'hello'를 리스트(list)로 변경
```

기적의 TIP

list란 C언어와 자바에서 배열과 같은 기능을 수행하는 군집형 자료형입니다.

기적의 TIP

input 명령에서 숫자를 입력하면 파이썬은 자동으로 문자형으로 인식하기 때문에 7과 7을 결합(+)하여 77이 됩니다.

기적의 TIP

문자형 7은 int로 형 변환되어 숫자 7이 됩니다.

• 예제1

```
a=input('숫자를 입력하세요') #7을 입력
print(a+a)
```

결과	77
----	----

• 예제2

```
a=int(input('숫자를 입력하세요')) #7을 입력
print(a+a)
```

결과	14
----	----

2) 자료형 확인하기

- type 함수를 이용하면 자료형을 확인할 수 있다.

• 예제

```
a=input('숫자 하나를 입력하세요') # 5를 입력하고 [Enter]
b=int(a)
c=float(a)
print(type(a))
print(type(b))
print(type(c))
```

결과	<class 'str'> <class 'int'> <class 'float'>
----	---

3) 한 줄로 출력(print)하는 방법

① 콤마(,)를 이용하는 방법

```
>>> x=3
>>> y=5
>>> print(x, '과', y, '의 합은 ', x+y, '이다.')
3 과 5 의 합은 8 이다.
```

+ 더 알기 TIP

콤마(,)를 이용하면 콤마 오른쪽이 자동으로 사이 띄기가 됩니다.

② 형 변환을 이용하는 방법

```
>>> x=3
>>> y=5
>>> print(str(x)+'과 '+str(y)+'의 합은 '+str(x+y)+'이다.')
```

3과 5의 합은 8이다.

+ 더 알기 TIP

x와 y는 숫자이기 때문에 '+'를 이용해서 결합할 수 없습니다. x와 y를 문자형(str)로 변환한 후 결합해서 한 줄로 출력하는 방법입니다.

③ end=" (출력 끝에 붙는 문자열)

- print문에서 end=" 문을 이용하면 줄 바꿈이 없어지게 된다.

```
x=3
y=5
print(x,end='')
print('과 ',end='')
print(y,end='')
print('의 합은 ',end='')
print(x+y,end='')
print('이다.')
```

결과 3과 5의 합은 8이다.

+ 더 알기 TIP

- end = '출력 끝에 붙는 문자열'
예) print(3,5,end='a') → 3 5a
- sep = '출력할 값들 사이에 들어갈 구분 문자열'
예) print(3,5,10,sep='a') → 3a5a10

④ format() 매서드 사용

- 자리를 지정해주는 방식으로 가장 많이 사용하는 방식이다.

```
x=3
y=5
print('{}과 {}의 합은 {}이다.'.format(x,y,x+y))
```

결과 3과 5의 합은 8이다.

기적의 TIP

①번과 ②번은 실제로는 잘 사용하지 않는 방법이며, 다음에 배우는 ④번 방식을 많이 사용하고 있습니다.

04 연산자

1) 산술 연산자

① 산술 연산자 종류

연산자 종류	의미	연산자 종류	의미
$a + b$	a 더하기 b	$a // b$	a를 b로 나눈 몫(정수)
$a - b$	a 빼기 b	$a \% b$	a를 b로 나눈 나머지
$a * b$	a 곱하기 b	$a ** b$	a의 b 제곱
a / b	a 나누기 b(실수)		

+ 더 알기 TIP

파이썬에서 정수끼리의 연산 결과는 정수가 출력되지만, 나누기(/) 연산은 정수끼리의 연산이라고 하더라도 결과는 자동으로 실수가 됩니다.

② 산술 연산 예제

```
a=10          #a는 자동으로 정수형 변수가 되며 10을 넣는다.
b=5
c=2.0         #c는 자동으로 실수형 변수가 되며 2.0을 넣는다.
d=0.5
print(a+b)
print(a-b)
print(a*b)
print(a/b)
print(a//b)
print(a%b)
print(a**b)
```

결과	15
	5
	50
	2.0
	2
	0
	100000

2개의 연산에서 하나라도 실수가 있으면 결과는 무조건 실수가 됩니다.

수정 소스	결과
b=5	
c=2.0	
print(b+c)	→ 7.0
print(b-c)	→ 3.0
print(b*c)	→ 10.0
print(b/c)	→ 2.5
print(b//c)	→ 2.0
print(b%c)	→ 1.0
print(b**c)	→ 25.0

2) 관계 연산자와 논리 연산자

① 관계 연산자와 논리 연산자 종류

관계 연산	의미	논리 연산	연산자 의미
<	작다	x or y	둘 중 하나만 참이면 참
<=	작거나 같다	x and y	x, y 모두 참일 때만 참
>	크다	not x	x의 반대
>=	크거나 같다		
==	같다		
!=	같지 않다		

② 관계 연산과 논리 연산 예제

논리형 소스	결과
x = 10	
y = -10	
print(x > 0)	→ True
print(x < y)	→ False
print(x == 10)	→ True
print(x != y)	→ True
print(x > 0 or y > 0)	→ True
print(x > 0 and y > 0)	→ False
print(not x > 0)	→ False

기적의 TIP

'='는 대입 연산자, '=='는 비교 연산자입니다.

기적의 TIP

- `a << 3` → 왼쪽으로 3번 이동
- `a >> 3` → 오른쪽으로 3번 이동

3) 비트 연산자

① 비트 연산자 종류

연산자	의미	설명
<code>&</code>	and	두 개의 비트가 모두 1일 때만 1이고, 나머지는 0
<code>^</code>	xor	두 개의 비트가 서로 같으면 0, 서로 다르면 1
<code> </code>	or	두 개의 비트가 모두 0일 때만 0이고, 나머지는 1
<code>~</code>	not	0이면 1, 1이면 0
<code><<</code>	왼쪽 시프트	비트를 왼쪽으로 이동
<code>>></code>	오른쪽 시프트	비트를 오른쪽으로 이동

+ 더 알기 TIP

- 비트 연산을 하려면 먼저 2진수로 변환하고 수행해야 합니다.
- 비트 연산과 2진수 변환을 모르면 C언어 연산자를 다시 한번 학습하시기 바랍니다.

② 비트 연산 예제

논리형 소스		결과
<code>print(10 & 6)</code>	→	2
<code>print(10 6)</code>	→	14
<code>print(10 ^ 6)</code>	→	12
<code>print(10 << 1)</code>	→	20
<code>print(10 >> 1)</code>	→	5

4) 복합 연산자

① 복합 연산자 종류

연산자	의미	연산자	의미
<code>a += 10</code>	<code>a = a + 10</code>	<code>a *= 10</code>	<code>a = a * 10</code>
<code>a -= 10</code>	<code>a = a - 10</code>	<code>a /= 10</code>	<code>a = a // 10</code>

② 복합 연산 예제

```
a=10
a += 1
a -= 2
a *= 3
a /= 4
print(a)
```

결과 6

05 연산자 우선순위

1) 산술 연산자

우선순위	연산자	의미
1	$a ** b$	지수
2	$a * b, a / b$ $a // b, a \% b$	곱하기, 나누기 몫, 나머지
3	$a + b, a - b$	더하기, 빼기

2) 관계 연산자

우선순위	연산자	의미
같음	$<, <=$ $>, >=$ $==, !=$	작다(미만), 작거나 같다(이하) 크다(초과), 크거나 같다(이상) 같다, 같지 않다(다르다)

3) 논리 연산자

우선순위	연산자	의미
1	$\text{not } x$	x 가 참이면 거짓, 거짓이면 참
2	$x \text{ and } y$	x, y 모두 참이면 참, 나머지는 거짓
3	$x \text{ or } y$	x, y 모두 거짓이면 거짓, 나머지는 참

+ 더 알기 TIP

연산자 우선순위 예제

```
print(2*5>2+5 and not 3*3>10)
```

• 연산 순서

2*5	>	2+5	and	not	3*3	>	10
①	③	②	⑤	④	①	③	②

• 결과 : True

기적의 TIP

산술 연산자 > 관계 연산자 >
논리 연산자

01 다음 중 파이썬에서 변수 이름으로 사용할 수 없는 것은?

- ① total_sum ② _count
③ num1 ④ lnum

파이썬에서 변수 이름은 문자 또는 _로 시작할 수 있으며, 숫자로 시작할 수 없다.

02 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
x = "10"
y = 10
print(x + y)
```

- ① 20 ② "1010"
③ 에러 발생 ④ 10

문자열과 정수는 서로 더할 수 없다(TypeError 발생).

03 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
print(2 ** 3)
```

- ① 6 ② 8
③ 9 ④ 16

**는 거듭제곱을 구하는 연산자로, 2의 3제곱(2³)을 의미한다.

04 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
x = 5
x += 3
print(x)
```

- ① 5 ② 3
③ 8 ④ 15

x += 3은 x = x + 3의 뜻으로, 5인 x에 3을 더해 8이 된다.

05 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
print("A" * 3)
```

- ① A3 ② AAAA
③ AAA ④ 에러

문자열×정수는 반복 출력을 의미한다.

06 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
print("A", "B", "C", sep="-")
```

- ① A B C ② A-B-C
③ ABC ④ A,B,C

sep="삽입문자"는 문자 사이에 삽입문자가 삽입된다.

07 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
print("ABC", "ABC", sep="123", end="321")
```

- ① ABC ABC ② ABC123ABC
③ ABC123ABC321 ④ ABC321ABC123

• sep : 출력 값 사이에 들어갈 문자열
• end : 출력 마지막에 추가될 문자열(기본값은 줄바꿈 \n)

08 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
print(5//2)
```

- ① 2.0 ② 2
③ 2.5 ④ 1

• //는 정수 나눗셈(몫) 연산자로, 나눗셈 결과에서 소수점 이하를 버린다.
• 파이썬에서 / 연산은 항상 실수(float)를 반환하고, // 연산은 소수점을 버린 정수(int)를 반환한다.

Python 문자열 슬라이스와 군집 자료형

반출 태그 문자열 • 슬라이스 • 리스트 • 튜플 • 딕셔너리

01 문자열 슬라이스

1) 문자열 인덱스

① 문자열 인덱스 예제1

```
text='han'
print(text[0])
print(text[1])
print(text[2])
```

결과

h
a
n

- text='han'은 문자열을 자동으로 text 배열(리스트)에 한 글자씩 저장한다. 그래서 배열 값을 인쇄하면 위와 같이 한 글자씩 출력된다.
- 위 소스에 'print(text[3])'을 추가하면 test[3]은 존재하지 않기 때문에 에러가 발생한다.

② 문자열 인덱스 예제2

```
text='han'
print(text[-3])
print(text[-2])
print(text[-1])
```

결과

h
a
n

- 배열(리스트) 인덱스는 왼쪽부터 오른쪽으로는 0부터 시작되며, 오른쪽부터 왼쪽으로는 -1부터 시작한다.

h	a	n
text[0]	text[1]	text[2]
text[-3]	text[-2]	text[-1]

- 배열 인덱스 번호는 정수만 가능하며, 실수를 넣으면 에러가 발생한다.

기적의 TIP

배열의 시작은 0번째부터 시작합니다.

2) 문자열 슬라이스

- 형식 : text[시작인덱스:끝인덱스-1]

① 문자열 슬라이스 예제1

```
text='handaman nice'
print(text[0:5])      #0번째부터 4번째 문자 출력
print(text[5:8])      #5번째부터 7번째 문자 출력
print(text[5:])        #5번째부터 오른쪽 끝까지 출력
```

결과

```
handa
man
man nice
```

h	a	n	d	a	m	a	n		n	i	c	e
0	1	2	3	4	5	6	7	8	9	10	11	12
-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

② 문자열 슬라이스 예제2

```
text='handaman nice'
print(text[-8:-5])    #-8번째부터 -6번째 문자 출력
print(text[-8:])       #-8번째부터 오른쪽 끝까지 출력
```

결과

```
man
man nice
```

③ 문자열 슬라이스 예제3

```
text='handaman nice'
print(text[:5])        #처음부터 5-1번째인 4번째까지 가져옴(handa)
print(text[:])          #처음부터 끝까지 가져옴
print(text[0:11:2])     #0번째부터 2칸씩 건너서 11-1번째인 10번째까지 가져옴
print(text[::-1])       #문자열을 거꾸로 가져옴
```

결과

```
handa
handaman nice
hnaa i
ecin namadnah
```

3) 문자열 함수

① format(a, b, c, ...): 출력지정

```
text = 'abcde {} {}'  
print(text.format('ABC',123)) # {}에 'ABC'와 123을 넣어서 출력
```

결과	abcde ABC 123

② replace(a, b): 교체하기

```
text = 'abcde ABS ABC'  
print(text.replace('A','K')) # 'A'를 'K'로 교체
```

결과	abcde KBS KBC

③ split(): 자르기

```
text = 'abcde A/B/C A.B.C'  
a,b,c = text.split() # text에 입력된 문자를 공백 기준으로 자르기  
print(a)  
print(b)  
print(c)  
print()
```

```
a,b,c = text.split('/') # '/' 기준으로 자르기  
print(a)  
print(b)  
print(c)  
print()
```

```
a,b,c = text.split('.') # '.' 기준으로 자르기  
print(a)  
print(b)  
print(c)  
print()
```

결과	abcde A/B/C A.B.C abcde A B C A.B.C abcde A/B/C A B C

④ a.join() : 합치기

```
text = 'abcde'
print('/'.join(text)) # text에 입력된 각 문자 사이에 '/'를 넣기
```

결과	a/b/c/d/e
----	-----------

⑤ count() : 개수 확인하기

```
text = 'abcde ABC ABC'
print(text.count('a'))
print(text.count('A'))
print(text.count('1')) # text에 입력된 문자열에 1은 없어서 0 출력
```

결과	1
	2
	0

⑥ strip(a) / lstrip(a) / rstrip(a) : 제거하기

```
text = '   abcde   '
print(text.strip())
print(text.lstrip())
print(text.rstrip())
print()
text = '**ab**cde**'
print(text.strip('*'))
print(text.lstrip('*'))
print(text.rstrip('*'))
```

결과	abcde
	abcde
	abcde
	ab**cde
	ab**cde**
	abcde

+ 더 알기 TIP

- strip(*)은 양쪽 끝에 있는 *를 지운다.
- 중간에 있는 문자는 지울 수 없으며 lstrip은 왼쪽, rstrip은 오른쪽을 지운다.

⑦ find(a) / rfind(a) / index(a) / rindex(a) : 인덱스 찾기

```
text = 'ABC ABC'
print(text.find('A'))           # 왼쪽부터 'A'를 찾아서 인덱스 넘버 출력
print(text.rfind('A'))         # 오른쪽부터 'A'를 찾아서 인덱스 넘버 출력
print(text.index('A'))         # 왼쪽부터 'A'를 찾아서 인덱스 넘버 출력
print(text.rindex('A'))        # 오른쪽부터 'A'를 찾아서 인덱스 넘버 출력

print(text.find('a'))          # find는 없는 문자를 찾는 경우 -1 출력
print(text.rfind('a'))
print(text.index('a'))         # index는 없는 문자를 찾는 경우 에러 발생
print(text.rindex('a'))
```

결과

```
0
4
0
4
-1
-1
# substring not found 에러 메시지가 나타난다.
```

⑧ isalpha() / isdigit() / isalnum() / isupper() / islower() : 확인하기(True, False)

```
text1 = 'ABCabc123'
text2 = '123'
text3 = 'ABC'
text4 = 'abc'
text5 = 'ABC1'

print(text1.isalpha())        # text1의 값이 모두 영문자이면 참, 아니면 거짓
print(text2.isdigit())        # text2의 값이 모두 숫자이면 참, 아니면 거짓
print(text1.isalnum())        # text1의 값이 영문자와 숫자의 조합이면 참, 아니면 거짓
print(text3.isupper())        # text3의 영문자가 모두 대문자이면 참, 아니면 거짓
print(text3.islower())        # text3의 영문자가 모두 소문자이면 참, 아니면 거짓
print(text5.isupper())        # text5의 영문자가 모두 대문자이면 참, 아니면 거짓
```

결과

```
False
True
True
True
False
True
```

기적의 TIP

isupper()과 islower()은 모두 영문자가 아니어도 문자열에서 영문자가 모두 대문자이거나 소문자인지 판정합니다.

⑨ upper() / lower() : 대/소문자 만들기

```
text = 'ABCabc'
print(text.upper())
print(text.lower())
print(text[3].upper())
print(text[3])
```

결과

ABCABC
abcabc
A
a

⑩ zfill() : 자리수만큼 0으로 채우기

```
y='2020'
m='3'
d='1'
print(y.zfill(4))
print(m.zfill(4))
print(d.zfill(4))
```

결과

2020
0003
0001

02 군집 자료형

1) 리스트(list)

- 파이썬의 리스트는 C언어와 자바의 배열과 동일한 것으로 여러 개의 값을 하나의 이름으로 처리할 때 사용하는 배열이다.
- 리스트는 배열의 개수를 늘리거나 줄일 수 있어서 리스트를 선언할 때 크기를 적지 않는다.

① 형식

리스트 이름 = [값1, 값2, 값3 ...]

리스트 이름 = list([값1, 값2, 값3 ...])

+ 더 알기 TIP

```
li.index('c')          # 'c'의 인덱스 번호 찾기
li[2]='d'              # li[2]에 있던 'c'가 'd'로 변경
li.append('f')         # 리스트 마지막에 'f' 추가
li.insert(0,'aa')      # 0번 인덱스에 'aa'가 삽입되고 원래 0번 인덱스에 있던 값은
                        # 뒤로 한 칸씩 밀려남
li.remove('aa')        # 리스트에 있는 'aa'를 삭제하고 오른쪽에 있던 값은 앞으로 한 칸씩
                        # 이동됨(리스트에 없는 값은 예외 발생)
del li[2]              # 2번 인덱스의 값 삭제
'b' in li              # 리스트에 'b'가 있으면 True, 없으면 False
len(li)               # 리스트의 전체 개수
li.count('a')          # 리스트 값들 중에서 'a'의 개수 세기
```

② 예제1 - list 슬라이스

```
>>> li=['a', 'b', 'c', 'd', 'e']
>>> li
['a', 'b', 'c', 'd', 'e']
>>> li[2:4]          # 2번 인덱스부터 4 미만인 3 인덱스까지 슬라이스 한다.
['c', 'd']
```

📌 기적의 TIP

리스트 슬라이스는 앞에서 다룬 문자열 슬라이스와 사용법은 동일하지만, 문자열 슬라이스는 문자열의 결과를 수정할 수 없고 리스트 슬라이스는 수정이 가능하다는 차이점이 있습니다.

+ 더 알기 TIP

```
num=[1,2,3,4,5,9,7,6,8,10]
sum(num)          # num 리스트 값들의 합계를 구함
min(num)          # num 리스트 값들의 최소값을 구함
max(num)          # num 리스트 값들의 최대값을 구함
num.reverse()     # num 리스트 값들을 역순으로 배열
num.sort()        # num 리스트 값들을 오름차순 정렬
num.sort(reverse=True) # num 리스트 값들을 내림차순 정렬
num.sort(reverse=False) # num 리스트 값들을 오름차순 정렬
```

③ 예제2

```
list1 = [1, 2, 3]
list2 = [4, 5]
list1.insert(2, list2) # list2를 하나의 요소로 인식하여 리스트 전체를 삽입한다.
print(list1)
```

결과	[1, 2, [4, 5], 3]	# 리스트 안에 리스트가 삽입됨
----	-------------------	-------------------

2) 딕셔너리(Dictionary)

- 리스트의 인덱스처럼 자동으로 키 값이 정해지는게 아니라 키 값을 사용자가 직접 지정할 수 있는 군집 자료형이다.
- 딕셔너리는 중괄호({})를 사용한다.

① 형식

```
dic = {키1:값, 키2:값, 키3:값}
se1={} # 비어있는 딕셔너리를 만든다.
se2=dict() # dict() 함수를 이용하여 비어있는 딕셔너리를 만든다.
```

② 예제1 - 딕셔너리 기본

```
>>> dic={'kor':80,'eng':90,'mat':77}
>>> dic
{'kor': 80, 'eng': 90, 'mat': 77}
>>> dic['kor'] # 딕셔너리의 값에 접근할 때는 대괄호로 구분한다.
80
>>> dic['kor'] = 85 # dic['kor'] 값을 85로 변경한다.
>>> dic['kor']
85
>>> dic['sci'] = 92 # 딕셔너리에 새로운 키와 값을 추가한다.
>>> dic
{'kor': 85, 'eng': 90, 'mat': 77, 'sci': 92}
```


③ 예제2 – 딕셔너리 활용

```
>>> dic={'kor':88,'eng':90,'mat':77,'sci':92}
>>> del dic['mat']           # 특정 키와 값을 삭제한다.
>>> dic
{'kor': 88, 'eng': 90, 'sci': 92}
>>> dic.clear()             # 딕셔너리 전체를 삭제한다.
>>> dic
{}
>>> dic={'kor':88,'eng':90,'mat':77,'sci':92}
>>> 'eng' in dic             # 키 값을 확인한다. 있으면 True 없으면 False
True
>>> 'com' in dic
False
>>> len(dic)                 # 전체 개수(키와 값 한 쌍으로 계산)
4
>>> list(dic.keys())         # 키를 리스트로 만든다.
['kor', 'eng', 'mat', 'sci'] # dic가 리스트로 변경된 것은 아님
```

3) 튜플(Tuple)

- 리스트와 사용법은 같지만, 값을 변경(추가/삭제 등)할 수 없는 열거형 집합이다.
- 튜플은 소괄호()를 사용한다.

① 형식

```
tu=()
```

```
tu=tuple()
```

② 예제1

```
tu=('a','b','c')
tu[0]           # 'a'
tu[2]           # 'c'
tu[2]='d'       # 튜플은 변경할 수 없기 때문에 에러 발생
```

③ 예제2

```
tu=('a','b','c')
tu.index('c')   # 'c'의 위치 찾기 → 2
'b' in tu       # 'b'가 있는지 확인하기 → True
len(tu)         # 전체 개수 → 3
tu.count('a')   # 'a'의 개수 → 1
```

4) 세트(Set)

- 순서가 없고 중복이 허용되지 않는 군집 자료형이다.
- 세트는 중괄호({})를 사용하며 순서가 없기 때문에 인덱스 번호를 사용할 수 없다.

① 형식

```
se1={2,4,6,8}    # 순서에 상관없이 세트를 만든다.  
se2=set()        # 비어있는 세트를 만든다.  
se3={}           # 딕셔너리가 만들어짐. 비어있는 세트는 set() 함수를 사용해야 한다.
```

② 예제

```
>>> a={2,4,6,8}  
>>> a  
{8, 2, 4, 6}    # 순서는 랜덤하게 만들어진다.  
>>> b={2,4,2,1,2,3}  
>>> b  
{1, 2, 3, 4}    # 중복이 없어지고 하나씩만 남는다.  
>>> a[0]          # 세트는 순서가 없어서 인덱스 번호를 사용할 수 없다.  
TypeError 발생  # list2를 하나의 요소로 인식하여 리스트 전체를 삽입한다.
```

01 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
s = "PYTHON"
print(s[-1])
```

- ① P
- ② O
- ③ N
- ④ H

음수 인덱스는 오른쪽 끝부터 -1로 시작하면서 1씩 감소한다.

02 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
s = "PYTHON"
print(s[1:4])
```

- ① YTH
- ② YTH
- ③ YTHO
- ④ Y

- 슬라이스 규칙 : [시작:끝]으로 끝 인덱스는 포함하지 않는다.
- print(s[:끝]) → 시작이 생략되면 0부터 시작하고, 둘 다 생략하면 전체를 가져온다.

03 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
s = "ABC"
print(s[::-1])
```

- ① ABC
- ② BCA
- ③ CAB
- ④ CBA

[::-1]는 문자열을 거꾸로 뒤집는다.

04 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
s = "PYTHON"
print(s[-3:])
```

- ① PY
- ② PYTH
- ③ HON
- ④ THON

[-3:]은 뒤에서 세 번째 문자 'H'부터 오른쪽 끝까지 가져온다.

05 다음 중 문자열 슬라이싱(string slicing)과 리스트 슬라이싱(list slicing)의 차이점으로 옳은 것은?

- ① 문자열 슬라이싱은 인덱스를 사용할 수 없고, 리스트 슬라이싱은 사용할 수 있다.
- ② 문자열 슬라이싱 결과는 리스트이고, 리스트 슬라이싱 결과는 문자열이다.
- ③ 문자열은 슬라이싱 결과를 수정할 수 없고, 리스트는 수정할 수 있다.
- ④ 리스트 슬라이싱은 음수 인덱스를 사용할 수 없다.

문자열과 리스트의 슬라이스 문법은 같지만 문자열 슬라이스는 결과를 수정할 수 없고, 리스트는 수정이 가능하다는 차이가 있다. 즉, 파이썬에서 문자열(str)은 불변(immutable) 객체이고, 리스트(list)는 가변(Mutable) 객체라는 점이 중요하다.

06 Python 데이터 타입 중 시퀀스(Sequence) 데이터 타입에 해당하며 다양한 데이터 타입들을 주어진 순서에 따라 저장할 수는 있으나 저장된 내용을 변경할 수는 없는 것은?

- ① 복소수(Complex) 타입
- ② 리스트(List) 타입
- ③ 사전(Dict) 타입
- ④ 튜플(Tuple) 타입

오답 피하기

- 리스트 : 저장된 내용을 변경할 수 있다.
- 딕셔너리 : 키-값 구조로 시퀀스(순서)가 아니다.

07 다음 중 파이썬 시퀀스(Sequence) 자료형이 아닌 것은?

- ① 리스트(list)
- ② 튜플(tuple)
- ③ 문자열(str)
- ④ 사전(dict)

dict(딕셔너리)는 카-값 구조로 시퀀스(순서)가 아니다.

08 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
arr1 = [0, 1, 2, 3, 4]
print(arr1[1:4])
```

- ① [0, 1, 2, 3]
- ② [1, 2, 3]
- ③ [2, 3, 4]
- ④ [1, 2, 3, 4]

리스트[시작인덱스:종료인덱스]는 시작 인덱스부터 종료 인덱스 전까지 출력한다. 이때 끝 인덱스는 포함되지 않는다.

Python 제어문과 반복문

반출 태그 제어문 • 반복문

01 제어문

1) if문

① 단일 if

• 형식

```
if 조건:
    실행문           # 조건이 참인 경우
```

• 예제

```
num=int(input('숫자 하나 입력:')) # 5를 입력
if num>0:
    print('{}은(는) 양수입니다.'.format(num))
```

결과	숫자 하나 입력:5 5은(는) 양수입니다.
-----------	----------------------------

② if ~ else

• 형식

```
if 조건:
    실행문1           # 조건이 참인 경우
else:
    실행문2           # 조건이 거짓인 경우
```

• 예제

```
num=int(input('숫자 하나 입력:')) # 5를 입력
if num%2==0:
    print('{}은(는) 짝수입니다.'.format(num)) # 참인 경우
else:
    print('{}은(는) 홀수입니다.'.format(num)) # 거짓인 경우
```

결과	숫자 하나 입력:5 5은(는) 홀수입니다.
-----------	----------------------------

③ if ~ elif

- 형식

```
if 조건1:
    실행문1          # 조건1이 참인 경우 실행
elif 조건2:
    실행문2          # 조건1이 거짓이고 조건2가 참인 경우 실행
elif 조건3:
    실행문3          # 조건2가 거짓이고 조건3이 참인 경우 실행
...
```

- 예제

```
age=int(input('나이 입력:'))      # 23을 입력한다.
if age<=7:
    print('유아입니다.')          # 참인 경우
elif age<=19:
    print('청소년입니다.')        # 거짓인 경우 첫번째 질문
elif age>=20:
    print('성인입니다.')          # 거짓인 경우 두번째 질문
```

결과	나이 입력:23 성인입니다.
-----------	--------------------

④ if ~ elif ~ else

- 형식

```
if 조건1:
    실행코드(참인 경우)
elif 조건2:
    실행코드(조건1이 거짓인 경우 다시 질문)
elif 조건3:
    실행코드(조건2가 거짓인 경우 다시 질문)
...
else:
    실행코드(모든 조건이 거짓인 경우 실행)
```

• 예제1

```
text=input('알파벳 입력:')
if text.isupper():
    print('대문자')           # 참인 경우
elif text.islower():
    print('소문자')           # 거짓인 경우 첫번째 질문
else:
    print('대/소문자 구분 불가능') # 모든 질문에 만족하지 않는 경우
```

결과

알파벳 입력:abc
소문자

알파벳 입력:123
대/소문자 구분 불가능

알파벳 입력:Ac
대/소문자 구분 불가능

알파벳 입력:AB1
대문자

+ 더 알기 TIP

- text.isupper : text 변수에 입력받은 문자가 대문자이면 참, 아니면 거짓
- text.islower : text 변수에 입력받은 문자가 소문자이면 참, 아니면 거짓

• 예제2

```
age=int(input('나이 입력:'))
if age<=7:
    print('유아입니다.')
elif age<=19:
    print('청소년입니다.')
else:
    print('성인입니다.')
```

결과

나이 입력:20
성인입니다.

나이 입력:15
청소년입니다.

나이 입력:7
유아입니다.

2) 중첩 if문

- 예제 - 중첩 if문을 이용하여 청소년을 초,중,고등학생으로 분류하는 프로그램

```
age=int(input('나이 입력:'))
if age<=7:
    print('유아입니다.')
elif age<=19:
    print('청소년입니다.')
    if age<=13:
        print('초등학생')
    elif age<=16:
        print('중학생')
    else:
        print('고등학생')
elif age>=20:
    print('성인입니다.')
```

결과

나이 입력:13
청소년입니다.
초등학생

나이 입력:20
성인입니다.

나이 입력:17
청소년입니다.
고등학생

02 반복문 while

1) while

- 조건이 참인 동안 실행문이 반복되는 반복문이다.

① while문 - 기본

- 형식

```
while 조건:
    실행문
```


- 예제

```
print('a가 0보다 크거나 같으면 실행, 작으면 정지')
a=int(input('a:'))
while a>=0:
    print('실행')          # a값이 0 이상이면 반복해서 '실행'을 출력
    a=int(input('a:'))
```

결과	a가 0보다 크거나 같으면 실행, 작으면 정지
	a:3
	실행
	a:0
	실행
	a:-2

② while문 – 무한 반복

- 예제

```
a=10
while a>0:
    print('실행')          # [Ctrl]+[C]를 누르면 강제 중지된다.
```

결과	실행
	실행
	실행
	..

③ while문 – n번 반복

- 예제1

```
n=int(input('양수 n:'))          # 5를 입력하고 [Enter]
while n:                         # 1 이상의 숫자는 참, 0은 거짓입니다.
    print(n)
    n=n-1
```

결과	양수 n:5
	5
	4
	3
	2
	1

- 예제2 - 1~10까지 합계를 출력하시오.

```
n=1
sum=0
while n<=10:
    sum=sum+n
    n=n+1
print(sum)
```

결과	55
-----------	----

④ while문 - break

- 예제 - 1부터 1씩 증가하며 입력받은 수까지 출력하는 프로그램

```
num1=0
num2=int(input('n : '))

while True:                # 무한 반복(True 대신 1을 써도 가능)
    num1=num1+1
    print(num1)
    if num1==num2:
        break              # 반복문을 벗어난다.
```

결과	n : 5 1 2 3 4 5
-----------	--------------------------------

+ 더 알기 TIP

- continue : 해당 단계만 건너뛰고 다음 단계로 간다.
- break : 제어문이나 반복문을 중단하고 빠져나간다.
- pass : 아무 작업도 하지 않는다.

03 for문

- 특정 범위만큼 코드를 반복 실행하는 조건문이다.

1) range

- 숫자의 범위와 증감 값을 규칙적인 수들의 집합으로 만들어주는 함수로써 for문에서 가장 많이 사용되는 함수이다.
- 형식 및 예제

```
range(a)                # 0부터 a 바로 전까지
```

예 range(11) # 0부터 10까지 1씩 증가

```
range(a, b)             # a부터 b 바로 전까지
```

예 range(1, 11) # 1부터 10까지 1씩 증가

```
range(a, b, c)          # a부터 b 바로 전까지 c씩 증가 또는 감소
```

예 range(1, 11, 2) # 1부터 10까지 2씩 증가

range(10, 0, -1) # 10부터 1까지 1씩 감소

+ 더 알기 TIP

a, b, c는 모두 정수(int)만 가능합니다.

2) for문 형식 및 예제

```
for 변수 in 열거형:
```

```
    실행 코드           # 열거형 데이터를 하나씩 변수에 대입하며 실행
```

- 예제1 - 0~9까지 출력

```
for i in range(10):     # i는 0부터 9까지 1씩 증가하며 반복문을 실행
    print(i)
```

결과

```
0
1
2
..
8
9
```

• 예제2 – 1부터 입력받은 수까지 출력

```
n=int(input('n: '))    # 3을 입력하고 [Enter]
for i in range(1,n+1): # i는 1부터 3까지 반복
    print(i)
```

결과

```
n: 3
1
2
3
```

• 예제3 – 임의의 수 2개를 입력받아 작은 수부터 큰 수까지 1씩 증가하며 출력

```
a,b=map(int,input('a b: ').split()) # 5 7을 입력하고 실행, 7 5를 입력하고 실행
if a<b:
    for i in range(a,b+1):
        print(i)
else:
    for i in range(b,a+1):
        print(i)
```

결과

```
a b: 5 7
5
6
7

a b: 7 5
5
6
7
```

+ 더 알기 TIP

map() 함수

- input().split()을 통해 입력받은 값을 원하는 자료형으로 변환하여 순서대로 변수에 대입하는 함수입니다.
- map 예제 – 여러 개의 숫자를 입력받아 리스트에 저장하기

```
>>> li1 = list(map(int, input('숫자를 입력하시오. ').split()))
숫자를 입력하시오. 11 22 33
>>> li1
[11, 22, 33]
```

• 예제4 - 입력받은 수부터 1까지 역순으로 출력하는 프로그램

```
n=int(input('n: '))          # 5를 입력하고 [Enter]
for i in range(n,0,-1):      # 5부터 1까지 1씩 감소하며 반복
    print(i)
```

결과	n: 5
	5
	4
	3
	2
	1

• 예제5 - 입력받은 수까지 3의 배수를 출력하는 프로그램

```
n=int(input('n : '))        # 10을 입력하고 [Enter]
for i in range(1,n+1):
    if i%3==0:
        print(i)
```

결과	n : 10
	3
	6
	9

• 예제6 - 리스트의 인덱스 번호가 짝수인 리스트 값을 출력하는 프로그램

```
li=[7,5,4,1,9]
for i in range(len(li)):    # i는 0부터 li 리스트에 담긴 값-1 만큼 반복
    if i%2==0:              # 인덱스 변수 i와 2를 나눈 나머지가 0이니?
        print(li[i])
```

결과	7
	4
	9

• 예제7 - 리스트에 저장된 값이 짝수인 리스트 값을 출력하는 프로그램

```
li=[6,5,4,1,9,2]
for i in range(len(li)):    # i는 li 리스트 담긴 값을 처음부터 마지막까지 반복
    if li[i]%2==0:          # i번째 리스트 값과 2를 나눈 나머지가 0이니?
        print(li[i])
```

결과	6
	4
	2

• 예제8 – 대소문자를 분류하여 리스트에 저장하는 프로그램

```
li=list(input('문자 입력 : '))    # 공백 포함 모든 글자를 각각 리스트로 만든다.
up=[]                            # 대문자가 담기는 빈 리스트 만든다.
low=[]                           # 소문자가 담기는 빈 리스트 만든다.
for i in li:                     # li 리스트에 담긴 문자를 처음부터 마지막까지 반복
    if i.isupper():              # i 리스트 값이 대문자인가?
        up.append(i)            # 참인 경우 up 리스트에 추가한다.
    elif i.islower():            # i 리스트 값이 소문자인가?
        low.append(i)           # 참인 경우 low 리스트에 추가한다.
print(up)                        # 대문자를 출력한다.
print(low)                       # 소문자를 출력한다.
```

결과

```
문자 입력 : YounGjiN
['Y', 'G', 'N']
['o', 'u', 'n', 'j', 'i']
```

01 다음은 파이썬으로 만들어진 반복문 코드이다. 이 코드의 결과는?

```
while True:
    print('A')
    print('B')
    print('C')
    continue
    print('D')
```

- ① A, B, C 출력이 반복된다.
- ② A, B, C까지만 출력된다.
- ③ A, B, C, D 출력이 반복된다.
- ④ A, B, C, D까지만 출력된다.

- continue문을 만나 while True문으로 이동하면서 무한 반복한다.
- print('D')은 한 번도 실행되지 않는다.

02 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
a = [10, 20, 30, 40, 50]
hap = 0
for i in a[1::2]:
    hap += i
print(hap)
```

- ① 40 ② 50 ③ 60 ④ 70

- a[1::2] # 1번 인덱스부터 2칸씩 건너뛴
- 추출된 값 : 20, 40 / 결과 : 20 + 40 = 60

03 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
total_sum = 0
my_list = range(10)
for i in my_list:
    total_sum += i
print(total_sum)
```

- ① 10 ② 44 ③ 45 ④ 55

- range(n)은 0부터 n-1까지의 정수를 생성한다. 즉, range(10)은 0~9까지이다.
- for i in my_list: → i는 0부터 9까지 1씩 증가하며 10회 반복한다.
- 0부터 9까지의 합계 계산 : 0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 = 45

04 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
n = 10
cnt = 0
while True:
    if n % 2 == 0:
        cnt += 1
    n += 1
    if n > 20:
        break
print(cnt)
```

- ① 5 ② 6
- ③ 10 ④ 11

- n은 10부터 20까지 1씩 증가하면서 n과 2를 나눈 나머지가 0인 개수(짝수)를 구하는 프로그램으로 총 6개(10, 12, 14, 16, 18, 20)이다.

05 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
a = [10, 20, 30, 40, 50]
hap = 0

for i in a[-3:]:
    hap += i
print(hap)
```

- ① 60
- ② 90
- ③ 120
- ④ 150

- a[-3:] 슬라이싱은 뒤에서 세 번째 요소(-3)부터 리스트의 끝까지 추출한다는 의미이다.
- 추출된 값 : 30, 40, 50 / 결과 : 30 + 40 + 50 = 120

함수, try, match

빈출태그 함수 • 전역변수 • 지역변수 • 예외처리 • try

01 사용자 함수

1) 기본 형식

```
def 함수명(인수1, 인수2...):
    명령줄1
    명령줄2
    ....
    return 명령
```

+ 더 알기 TIP

def는 함수를 정의하는 명령어이고, 인수는 생략이 가능합니다.

2) 예제

① 예제1 - 리턴 값이 없는 함수

- 편집기에서 작성한 후 실행한 경우

```
def aa():                # 인수가 없는 함수 aa를 만든다.
    print('handaman')    # aa() 함수는 한 줄을 출력하는 함수이다.

def bb(x):               # 인수가 있는 함수 bb를 만든다. 인수(x)는 임의로 설정
    for i in range(x):
        print('hello~')

# 함수 호출하기
re1=aa()
re2=bb(5)
```

결과

```
handaman
hello~
hello~
hello~
hello~
hello~
```


- 셀 화면에서 실행한 경우

```
>>> re1
>>> re2
>>>
```

+ 더 알기 TIP

아무런 결과가 없는 이유는 리턴 값이 없기 때문입니다.

② 예제2 – 리턴 값이 있는 함수

- 편집기에서 작성한 후 실행한 경우

```
def cc():                # 인수 없는 함수 cc를 만든다.
    n=int(input('n:'))   # 값을 입력받아 정수로 변환하여 n에 넣는다.
    print(n*2)           # 입력된 값에 2를 곱하여 출력한다.
    return n*2           # cc() 함수의 결과에 n*2 의 결과를 리턴한다.

def dd(x,y):             # 인자값이 있는 함수 dd를 만든다.
    print(x*y)           # 인자값 2개를 서로 곱하여 출력한다.
    return x*y           # 인자값 2개를 서로 곱한 결과를 dd 함수의 결과로 리턴한다.

# 함수 호출하기
re3=cc()
re4=dd(3,5)
```

결과	n:5	# 5를 입력하고 [Enter]
	10	
	15	

- 셀 화면에서 실행한 경우

```
>>> re3
10
>>> re4
15
>>>
```

+ 더 알기 TIP

re3, re4는 리턴 결과를 저장하고 있어서 위 결과처럼 출력됩니다.

③ 예제3 - 1부터 입력받은 수까지의 합계를 구하는 프로그램

```
def sum_func(n):
    s = 0
    for x in range(1, n+1):
        s = s + x
    return s
print("임의의 숫자를 입력하세요. ")
n = int(input())
print("1부터 " + str(n) + "까지의 합계는")
print(str(sum_func(n)) + "입니다. ")
```

결과

임의의 숫자를 입력하세요.
10
1부터 10까지의 합계는
55입니다.

02 전역변수와 지역변수

1) 전역변수

- 프로그램 전체에서 사용이 가능한 변수를 전역변수라 한다.
- 함수 호출 이전에 함수 밖에서 선언해야 한다.

2) 지역변수

- 함수 안에서 선언하고, 함수 안에서만 사용되는 변수를 지역변수라 한다.

3) 전역변수와 지역변수 예제

① 예제1

```
def func_a():
    num = 1                # func_a()에서만 사용하는 지역변수
    num = num + a          # 1인 num과 10인 a를 더해 num은 11이 된다.
    print(num)
a=10                      # 함수 호출 전에 a를 선언해서 a는 전역변수이다.
func_a()                  # func_a()를 호출하여 11을 출력한다.
print(a)                  # 전역변수 a에 담겨있는 10을 출력한다.
print(num)                # NameError: name 'num' is not defined 에러가 발생됨.
```

결과

10
11
NameError: name 'num' is not defined

② 예제2

```
def func_a():
    num = 1
    num = num + a
    print(num)

func_a()
a=10
print(a)
```

결과	# NameError : name 'a' is not defined
----	---------------------------------------

+ 더 알기 TIP

5번 줄의 "a=10"이 함수 호출 "func_a()" 아래에 있어서 3번 줄의 "num = num + a"를 수행할 수 없습니다. 왜냐하면 a에 무슨 값이 들어있는지 모르는 상태로 갔거든요.

③ 예제3

```
def func_a():
    a = 100          # 지역변수 a
    print('a in fun_a() : ',a)
a = 10              # 전역변수 a
print('a : ',a)
func_a()
print('a : ',a)
```

결과	a : 10 a in fun_a() : 100 a : 10
----	--

기적의 TIP

이름이 동일해도 사용되는 장소(함수)가 다르면 값이 달라질 수 있습니다. 익숙해질 수 있도록 코드를 분석하는 연습을 하세요.

03 내장 함수

1) math 모듈

① 형식

```
import math
math.함수
```

② 예제

```
import math
print(math.ceil(2.1))      # 올림
print(math.floor(2.1))    # 내림
print(math.factorial(5))  # 팩토리얼(1*2*3*4*5)
print(math.sqrt(4))       # 제곱근(루트)
print(math.pi)
```

결과	3
	2
	120
	2.0
	3.141592653589793

+ 더 알기 TIP

반올림(round)은 math 모듈을 import 하지 않아도 사용할 수 있습니다.

2) random 모듈

① 예제

```
>>> import random
>>> random.randint(1, 5)      # 1~5 내의 정수를 랜덤 생성
2
>>> random.random()          # 0 이상 1 미만의 랜덤한 소수 생성
0.15509616375335
>>> li=['a','b','c','d','e']  # 리스트 li 만들기
>>> random.choice(li)         # 리스트 값 중에 하나 랜덤 뽑기
'e'
>>> random.sample(li, 3)      # 리스트에서 랜덤하게 3개 뽑기
['b', 'd', 'a']
>>> random.shuffle(li)        # 리스트 값을 랜덤하게 섞기
['a', 'c', 'e', 'b', 'd']
```

04 try 문

1) 기본 형식

```
try:
    실행문1      # 실행문1이 에러가 없으면 실행한다.
except:
    실행문2      # try에서 에러가 발생되면 실행한다.
else:
    실행문3      # try에서 에러가 없으면 실행한다.
finally:
    실행문4      # 무조건 실행한다.
```

+ 더 알기 TIP

except가 생략되면 else는 실행되지 않습니다.

2) 예제

① except가 생략된 경우

```
try:
    print("A")
else:
    print("B")
finally:
    print("C")
```

결과	A
	C

② try에서 에러가 발생한 경우

```
try:
    print(1 / 0)      # 0으로 나눔 → 에러 발생
except:
    print(2)
finally:
    print(3)
```

결과	2
	3

③ try에서 에러가 발생하지 않은 경우

```
try:
    print(10 / 2)      # 에러가 발생하지 않음
except:
    print("에러")
else:
    print("정상")      # 에러가 없는 경우 실행
finally:
    print("끝")        # 무조건 실행
```

	5.0	# 정수 간의 나눗셈도 실수로 처리됨
결과	정상	
	끝	

05 match 문

- 기존의 if / elif를 더 깔끔하고 구조적으로 표현한다.
- match 값과 정확하게 일치하는 case를 찾아 실행문을 실행한다.
- case _ : match 값과 case 값이 모두 다른 경우 실행하는 것으로 'case _'이 없으면 아무 것도 실행되지 않을 수 있다.

1) 기본 형식

```
match 값:
    case 패턴1:      # 비교할 값
        실행문1      # 비교할 값과 같으면 실행문1을 실행한다.
    case 패턴2:
        실행문2
    case _:          # 비교할 값과 같은 경우가 없는 경우 실행문 3을 실행한다.
        실행문3
```

2) 예제

① 예제1

```
num = 2
match num:
    case 1:
        print("1입니다")
    case 2:
        print("2입니다")
    case 3:
        print("3입니다")
    case _:
        print("해당 없음")
```

결과	2입니다.
----	-------

② 예제2

```
my_list = [1, "2", 3, "4"]

for item in my_list:                # item은 0번째 배열부터 끝날 때까지 반복한다.
    match item:
        case "1":
            print("A", end='')
        case 2:
            print("B", end='')
        case 3:
            print("C", end='')
        case "4":
            print("D", end='')
        case _:
            print("X", end='')
```

결과	XXCD
----	------

+ 더 알기 TIP

- 1과 "1", "2"와 2는 자료형이 달라서 X를 2회 출력하고, 3에 해당하는 case 3문을 실행하여 C를 출력, "4"에 해당하는 case 4문을 실행하여 D를 출력한다.
- end="": 줄바꿈 하지 말고 이어서 출력한다.

01 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
my_list = ["1", "2", "3"]
match my_list[-3]:
    case "1":
        print(1, end="")
    case "2":
        print(2, end="")
    case "3":
        print(3, end="")
    case _:
        print("sit", end="")
```

- ① 1
② 2
③ 3
④ sit

"1"	"2"	"3"
my_list[0]	my_list[1]	my_list[2]
my_list[-3]	my_list[-2]	my_list[-1]

02 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
def f(x):
    x = x + 1
    return x
a = 3
f(a)
print(a)
```

- ① 3
② 4
③ 5
④ 오류 발생

x는 지역변수이며 함수 호출 후 반환 값을 저장하지 않았기 때문에 전역변수 a의 값은 변하지 않는다.

03 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
my_list = ["1", 2, "3"]
for item in my_list:
    match item:
        case 1:
            print(1, end="")
        case 2:
            print(2, end="")
        case 3:
            print(3, end="")
        case _:
            print(0, end="")
```

- ① 123
② 120
③ 020
④ 000

요소(item)	데이터 타입	매칭 결과	출력값
"1"	String	case _ (불일치)	0
2	Integer	case 2 (일치)	2
"3"	String	case _ (불일치)	0

04 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
try:
    print(1, end="")
    print(2, end="")
except:
    print(3, end="")
finally:
    print(4, end="")
```

- ① 123
② 134
③ 12
④ 124

- try → 정상 실행되면 except 건너뛰
- finally는 무조건 실행
- end="" → 줄바꿈 없이 출력

05 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
def sum(k):  
    k += 10  
    print(k)  
k = 5  
sum(k)  
print(k)
```

① 15

15

② 5

15

③ 15

5

④ 10

5

• k = 5, sum(k)	# 전역 변수 k에 5를 주고 sum(k)를 호출한다.
• k += 10, print(k)	# k=k+10을 실행하여 15를 출력하고 복귀한다.
• print(k)	# 전역 변수에 담겨있는 5를 출력한다.



이기적 강의는
무조건 0원!

이기적 영진닷컴

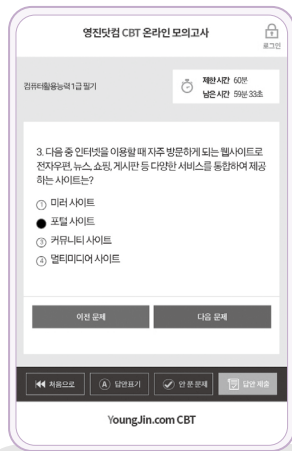


공부하다가
궁금한 사항은?

이기적 스터디 카페



2026년 제1회 기출복원문제



◀ 접속

CBT 온라인 문제집

- ① QR 코드 찍기(PC는 홈페이지 접속)
- ② 랜덤 모의고사 무료 응시
- ③ 풀이 후 자동 채점
- ④ 해설 즉시 확인 가능

프로그래밍기능사 필기 2026년 제1회 기출복원문제

01 html에서 쌍따옴표를 뜻하는 것은?

- ① "
- ②
- ③ <
- ④ >

02 css에서 규칙을 적용할 요소를 선택하는 것을 의미하는 것은?

- ① 선택자
- ② 클래스
- ③ 선언부
- ④ 속성

03 다음 중 커널의 역할로 옳지 않은 것은?

- ① 물리적 자원인 CPU 제공
- ② 물리적 자원인 메모리 제공
- ③ 입출력 관리
- ④ 명령어 저장

04 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
arr = [1, 2, 3, 4, 5]
arr[0:2]
```

- ① [1, 2, 3, 4, 5]
- ② [1, 3, 5]
- ③ [2, 4]
- ④ [1, 4, 5]

05 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
print("123", "123", sep="hello")
```

- ① 123 123
- ② 123hello123
- ③ 123,123
- ④ hello123123

06 다음 중 시각적으로 작동하는 사용자 인터페이스(UI)는 무엇인가?

- ① CUI
- ② GUI
- ③ NUI
- ④ VUI

07 다음 중 1:1, 1:N, N:N 등의 관계를 표현할 수 있으며 데이터를 2차원 테이블 형태로 표현하는 데이터 모델은?

- ① 관계형
- ② 계층형
- ③ 망형
- ④ 객체지향형

08 다음 중 IP 네트워크에서 IP 주소를 MAC 주소로 변환하는 프로토콜은 무엇인가?

- ① ARP
- ② RARP
- ③ DNS
- ④ SNMP

09 자바 프로그램을 백그라운드에서 실행하고 싶을 때 명령어 뒤에 붙이는 기호로 옳은 것은?

- ① &
- ② ?
- ③ !
- ④ <<

10 다음 중 IPv4 B 클래스의 기본 서브넷 마스크로 옳바른 것은?

- ① 255.0.0.0
- ② 255.255.0.0
- ③ 255.255.255.0
- ④ 255.255.255.255

11 OSI 7 계층 중 보안을 위해 데이터의 암호화와 해독을 수행하고 효율적인 전송을 위해 필요에 따라 압축과 전개를 수행하는 계층은?

- ① 전송 계층(Transport Layer)
- ② 표현 계층(Presentation Layer)
- ③ 응용 계층(Application Layer)
- ④ 데이터 링크 계층(Data Link Layer)

12 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
try:
    print(1)
else:
    print(2)
finally:
    print(3)
```

- ① 123
- ② 13
- ③ 23
- ④ 3

13 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
data = ["1", 2, "2"]
x = data[1]

match x:
    case 1:
        print(1)
    case 2:
        print(2)
    case "2":
        print(3)
    case _:
        print(0)
```

- ① 1
- ② 2
- ③ 3
- ④ 0

14 다음 중 FTP(File Transfer Protocol)의 특징으로 옳지 않은 것은?

- ① 3-핸드셰이크로 다방향 통신을 한다.
- ② 전송 계층 프로토콜을 사용한다.
- ③ 파일 전송을 위한 프로토콜이다.
- ④ 값 검증을 통해 전송의 정확성을 보장한다.

15 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
sum = 0
cnt = 0
for i in range(1, 100):
    sum += i
    cnt += 1
    if sum >= 20:
        break
print(cnt)
```

- ① 4
- ② 5
- ③ 6
- ④ 7

16 다음 중 DDL(Data Definition Language)에 속하지 않는 것은 무엇인가?

- ① DROP
- ② CREATE
- ③ ALTER
- ④ DELETE

17 다음 SQL문은 학생 테이블에서 학번이 NULL인 자료를 검색하는 문장이다. 정상적으로 실행되기 위해 빈칸에 들어갈 가장 알맞은 것은?

```
SELECT *
FROM 학생
WHERE 학번 ( ) NULL;
```

- ① IS
- ② HAS
- ③ NOT
- ④ IS NOT

18 다음 중 IPv4 주소에서 B 클래스의 시작 비트 값으로 올바른 것은?

- ① 00 ② 10
- ③ 01 ④ 11

19 다음 중 블랙박스 테스트 기법에 해당하지 않는 것은 무엇인가?

- ① 동등 분할
- ② 경계값 분석
- ③ 오류 예측
- ④ 기초경로 검사

20 리눅스에서 읽기 · 쓰기 · 실행 권한을 사용자(User), 그룹(Group), 기타(Other) 모두에게 부여하는 명령어는?

- ① chmod 777
- ② chmod 755
- ③ chmod 644
- ④ chmod 754

21 다음 중 교착상태가 발생하기 위한 조건이 아닌 것은 무엇인가?

- ① 상호 배제
- ② 점유와 대기
- ③ 환경(순환) 대기
- ④ 선점

22 다음 중 출시 전에 일부 사용자들이 실제 사용자 입장에서 사용성 및 완성도를 평가하는 테스트는 무엇인가?

- ① 알파 테스트(Alpha Test)
- ② 베타 테스트(Beta Test)
- ③ 검증(Verification)
- ④ 확인(Validation)

23 다음 중 선입선출(FIFO, First In First Out) 방식으로 자료를 처리하는 자료구조는 무엇인가?

- ① 트리
- ② 큐
- ③ 데크
- ④ 스택

24 다음 중 학생 테이블에서 나이가 23세 이상이고 29세 이하인 학생을 검색하는 올바른 SQL문은 무엇인가?

- ① SELECT * FROM 학생 WHERE 나이 >= 23 AND 나이 <= 29;
- ② SELECT * FROM 학생 WHERE 나이 > 23 AND 나이 < 29;
- ③ SELECT * FROM 학생 WHERE 나이 >= 23 AND 나이 < 29;
- ④ SELECT * FROM 학생 WHERE 나이 > 23 AND 나이 <= 29;

25 다음 파이썬 코드의 실행 결과로 옳은 것은 무엇인가?

```
total_sum = 0
my_list = range(10)
for i in my_list:
    total_sum += i
print(total_sum)
```

- ① 10
- ② 45
- ③ 50
- ④ 55

26 데이터베이스 설계 순서가 올바른 것은?

- ① 논리적 설계 → 개념적 설계 → 물리적 설계
- ② 물리적 설계 → 개념적 설계 → 논리적 설계
- ③ 개념적 설계 → 물리적 설계 → 논리적 설계
- ④ 개념적 설계 → 논리적 설계 → 물리적 설계

27 다음 중 데이터베이스 스키마를 3단계로 분류했을 때 해당하지 않는 것은 무엇인가?

- ① 외부 스키마
- ② 내부 스키마
- ③ 개념 스키마
- ④ 관계 스키마

28 다음 중 현재 시스템에서 실행 중인 프로세스 정보를 확인하는 명령어는 무엇인가?

- ① ps
- ② cd
- ③ mkdir
- ④ rm

29 다음 중 데이터베이스 관리 시스템(DBMS)의 장점이 아닌 것은?

- ① 데이터 중복 및 불일치 최소화
- ② 데이터 관리 비용 감소
- ③ 데이터의 보안 향상
- ④ 데이터의 일관성 및 무결성 유지

30 다음 중 OSI 7계층에서 네트워크 계층(Network Layer)의 기본 데이터 단위(PDU)는 무엇인가?

- ① 패킷(Packet)
- ② 세그먼트(Segment)
- ③ 비트(Bit)
- ④ 프로토콜(Protocol)

31 DevOps에서 “CI / CD”의 의미로 옳은 것은?

- ① 지속적 통합 / 지속적 배포
- ② 중앙 통합 / 코드 배포
- ③ 코드 인프라 / 클라우드 디버깅
- ④ 지속적 입력 / 지속적 데이터

32 윈도우에서 작업 관리자를 실행하는 단축키는?

- ①  + D
- ② Alt + Tab
- ③ Ctrl + Shift + Esc
- ④  + E

33 윈도우에서 휴지통을 거치지 않고 파일을 삭제하는 단축키는?

- ① Ctrl + Delete
- ② Alt + Delete
- ③  + Delete
- ④ Shift + Delete

34 다음 중 자바에서 주석으로 사용되지 않는 것은 무엇인가?

- ① //
- ② /* */
- ③ <!-- -->
- ④ /** */

35 UI(사용자 인터페이스)에서 도움말을 제공하는 것과 가장 관련 있는 것은?

- ① 직관성
- ② 학습성
- ③ 이식성
- ④ 효율성

36 HTML에서 문서의 구조를 설계하는 데 사용되지 않는 태그는?

- ① <html>
- ② <body>
- ③ <head>
- ④

37 다음 중 CASCADE 구문의 의미로 옳은 것은 무엇인가?

- ① 테이블의 튜플(레코드)만 제거한다.
- ② 테이블 자체를 제거한다.
- ③ 참조하는(연관된) 테이블의 데이터도 함께 제거한다.
- ④ 참조당하는 테이블만 제거한다.

38 릴레이션을 구성하는 모든 튜플에 대해 유일성은 만족하지만, 최소성은 만족하지 못하는 것은 무엇인가?

- ① 후보키 ② 기본키
- ③ 슈퍼키 ④ 대체키

39 트랜잭션(Transaction)이 실패한 후 ROLLBACK을 수행했을 때의 트랜잭션 상태로 올바른 것은 무엇인가?

- ① committed
- ② active
- ③ partially committed
- ④ failed

40 다음 중 메모리를 프로그램에 배정하는 작업을 의미하는 용어는?

- ① Deallocation
- ② Allocation
- ③ Compilation
- ④ Initialization

41 다음 중 CPU 사용 시간을 분할하여 여러 사용자가 동시에 사용하는 것처럼 처리하는 방식은?

- ① Time Sharing
- ② Batch Processing
- ③ Real Time Processing
- ④ Multiprocessing

42 다음 중 도메인(Domain)에 대한 설명으로 옳바른 것은?

- ① 튜플들을 구분하는 범위
- ② 튜플들의 모임
- ③ 속성들의 모임
- ④ 한 속성을 구분하는 범위

43 다음 중 데이터베이스의 보안과 무결성(연속성) 유지를 위한 명령어가 아닌 것은?

- ① GRANT ② REVOKE
- ③ COMMIT ④ UPDATE

44 다음과 같은 Java의 2차원 배열이 있을 때, arr.length의 실행 결과로 옳은 것은?

```
int[][] arr = { {1, 2}, {1, 2, 3} };  
System.out.println(arr.length);
```

- ① 5 ② 2
- ③ 3 ④ 컴파일 오류

45 다음 중 자바(Java)에서 상속 시 하위 클래스에서 접근할 수 없는 접근 제어자는?

- ① public ② protected
- ③ default ④ private

46 다음 파이썬 코드의 실행 결과로 옳은 것은?

```
cnt = 0  
i = 10  
while True:  
    if i % 2 == 0:  
        cnt += 1  
    i += 1  
    if i > 20:  
        break  
print(cnt)
```

- ① 5 ② 6
- ③ 10 ④ 11

47 다음 중 LAN의 통신 단위(접근 방식)가 아닌 것은?

- ① 토큰 링(Token Ring) 방식
- ② 토큰 버스(Token Bus) 방식
- ③ FDDI 방식
- ④ CSMA/CD 방식

48 다음 중 파일의 내용을 출력하거나 확인하는 유닉스 명령어가 아닌 것은?

- ① grep ② more
- ③ file ④ cat

49 다음 SQL문은 주소 테이블의 주소명 필드 길이를 50글자로 수정하려는 것이다. () 안에 들어갈 알맞은 SQL 명령어는?

ALTER TABLE 주소 () 주소명
VARCHAR(50);

- ① ADD ② UPDATE
- ③ MODIFY ④ DROP

50 다음 중 프레임(Frame)의 시작과 끝을 구분하기 위해 사용하는 플래그(flag)의 올바른 주소값은?

- ① 11100111 ② 00011000
- ③ 10000001 ④ 00010001

51 다음 중 TCP와 비교한 UDP의 특징에 대한 설명으로 옳바르지 않은 것은?

- ① 오류 제어와 흐름 제어를 수행하지 않는다.
- ② 전송 속도가 빠르다.
- ③ 비연결형 프로토콜이다.
- ④ 신뢰성이 있다.

52 다음은 자바 인터페이스와 클래스에 대한 코드이다. 이 코드를 실행했을 때 hellohello 가 출력되도록 (A)에 들어갈 알맞은 키워드를 고르시오.

```
interface A {
    void print();
}

class B (A) {
    public void print() {
        for (int i = 0; i < 2; i++)
            System.out.print("hello");
    }
}
```

- ① extends
- ② implements
- ③ import
- ④ interface

53 다음 자바 프로그램에서 (빈칸) 안에 들어갈 알맞은 키워드를 고르시오.

```
class A {
    void greet() {
        System.out.println("A");
    }
}

class B (    빈칸    ) A {
    @Override
    void greet() {
        System.out.println("B");
    }
}

public class Main {
    public static void main(String[] args) {
        A obj3 = new B();
        obj3.greet();
    }
}
```

- ① implements
- ② extends
- ③ import
- ④ interface

54 다음 자바 프로그램을 실행했을 때 출력되는 결과로 옳은 것은?

```
class A {
    void greet() {
        System.out.println("A");
    }
}

class B extends A {
    @Override
    void greet() {
        super.greet();
        System.out.println("B");
    }
}

public class Main {
    public static void main(String[] args) {
        A obj = new B();
        obj.greet();
    }
}
```

- ① A
- ② B
- ③ A B
- ④ A
B

55 다음 중 데이터 전체를 디스크에 저장한 후, 외부에 안전하게 보관하는 백업 방식은 무엇인가?

- ① 소산백업
- ② 부분백업
- ③ 전체백업
- ④ 증분백업

56 다음 중 POP3의 기본 포트번호는?

- ① 21
- ② 25
- ③ 80
- ④ 110

57 다음 중 DHCP(Dynamic Host Configuration Protocol)에 대한 설명으로 옳은 것은?

- ① 네트워크 장비 간 데이터 전송 경로를 설정하는 프로토콜이다.
- ② 클라이언트가 서버에 파일을 전송하기 위한 프로토콜이다.
- ③ IP 주소를 포함한 네트워크 설정 정보를 자동으로 할당해 주는 프로토콜이다.
- ④ 웹 페이지를 요청하고 응답하는 프로토콜이다.

58 다음 중 서브넷 마스크 255.240.0.0을 적용했을 때 올바른 서브넷 네트워크 주소가 아닌 것은?

- ① 100.5.0.0
- ② 100.32.0.0
- ③ 100.16.0.0
- ④ 100.48.0.0

59 다음 중 동적 라우팅 프로토콜에 해당하지 않는 것은?

- ① OSPF
- ② BGP
- ③ STP
- ④ RIP

60 다음 SQL문에서 중복된 결과를 제외하고 조회하기 위해 (A)에 들어갈 가장 알맞은 것은?

```
SELECT 이름 FROM 학생 (A) SELECT 이름 FROM 졸업생;
```

- ① UNION ALL
- ② UNION
- ③ UNIQUE
- ④ DISTINCT

2026년 제1회 기출복원문제

- 01 ① 02 ① 03 ④ 04 ② 05 ②
 06 ② 07 ① 08 ① 09 ① 10 ②
 11 ② 12 ② 13 ② 14 ① 15 ③
 16 ④ 17 ① 18 ② 19 ④ 20 ①
 21 ④ 22 ② 23 ② 24 ① 25 ②
 26 ④ 27 ④ 28 ① 29 ② 30 ①
 31 ① 32 ③ 33 ④ 34 ③ 35 ②
 36 ④ 37 ③ 38 ③ 39 ④ 40 ②
 41 ① 42 ④ 43 ④ 44 ② 45 ④
 46 ② 47 ④ 48 ③ 49 ③ 50 ①
 51 ④ 52 ② 53 ② 54 ④ 55 ①
 56 ④ 57 ③ 58 ① 59 ③ 60 ②

01 ①

오답 피하기

- ② → 공백(줄바꿈 없는 공백)
- ③ < → <(작다, less than)
- ④ > → >(크다, greater than)

02 ①

css의 기본 구조

```
선택자 {
  속성: 값;
}
```

- 선택자: CSS 규칙을 어떤 HTML 요소에 적용할지 선택하는 부분
- 클래스: 여러 요소에 공통 스타일을 적용하기 위해 사용하는 선택자의 한 종류
- 선언부: {} 안에 들어가는 속성: 값 전체
- 속성: 색상, 글꼴, 크기 등 스타일의 종류

+ 또 다른 기술

CSS에서 가장 기본적인 절대 길이의 단위는?

정답 px

03 ④

커널은 운영체제의 핵심(Core)으로, 하드웨어와 응용 프로그램 사이에서 자원 관리와 제어를 담당하며, CPU 스케줄링, 메모리 할당 및 관리, 입출력 장치 등의 제어를 담당한다.

오답 피하기

명령어 저장은 기억장치에 저장된다.

04 ②

리스트[시작:끝:간격], arr[0:2]는 인덱스 0번부터 끝까지 2칸씩 건너뛰며 선택한다.

05 ②

print() 함수에서 sep는 출력할 값들 사이에 들어갈 구분 문자열을 의미한다.

06 ②

GUI(Graphical UI): 아이콘, 버튼, 창 등 시각적 요소로 조작

오답 피하기

- CUI(Character UI): 문자(텍스트) 기반 인터페이스
- NUI(Natural UI): 터치, 제스처, 동작 등 자연스러운 입력
- VUI(Voice UI): 음성으로 조작

07 ①

관계형: 2차원 테이블(행·열), 1:1, 1:N, N:N 관계 표현 가능

오답 피하기

- 계층형: 트리 구조, 1:N 관계만 가능
- 망형: 그래프 구조, N:N은 가능하지만 2차원 테이블 아님
- 객체지향형: 객체 단위 저장, 클래스 기반

08 ①

ARP: IP 주소 → MAC 주소 변환

오답 피하기

- RARP: MAC 주소 → IP 주소 변환
- DNS: 도메인 이름 → IP 주소 변환
- SNMP: 네트워크 장비 감시 및 관리

09 ①

리눅스(유닉스) 환경에서 자바 프로그램을 실행할 때 명령어 뒤에 &를 붙이면 백그라운드 실행되고 터미널은 바로 다른 명령을 입력할 수 있는 상태가 된다.

10 ②

클래스	선두 비트	서브넷 마스크	주소 범위
A 클래스	0	255.0.0.0	1.0.0.0 ~ 126.0.0.0
B 클래스	10	255.255.0.0	128.0.0.0 ~ 191.255.0.0
C 클래스	110	255.255.255.0	192.0.0.0 ~ 223.255.255.0
D 클래스	1110		224 ~ 239
E 클래스	1111		240 ~ 255

11 ②

표현 계층은 데이터의 암호화, 압축, 코드변환, 구문 검색 기능을 수행한다.

12 ②

try ~ except ~ else ~ finally 구조의 실행 순서

- try 블록 실행
- try에서 에러 발생 → except 실행
- 에러가 없는 경우 → else 실행(except가 없으면 else는 실행하지 않음)
- finally는 무조건 실행

13 ②

- x에 data[1]에 있는 숫자형 2를 넣는다.
- match ~ case에서 match는 '값과 자료형이 모두 일치'하는 case 문을 찾아 실행한다.
- "2"는 문자형이고 2는 숫자형 자료이기 때문에 "case 2:" 문을 실행한다.
- 만약, 일치하는 자료가 하나도 없는 경우에는 "case _:" 문을 실행한다.

14 ①

- 3-핸드셰이크는 TCP의 연결 설정 과정이며, FTP는 다방향 통신을 하지 않는다.
- FTP는 응용계층 프로토콜이며 전송계층 프로토콜(TCP)을 사용하므로 전송의 신뢰성이 보장된다.

+ 또 다른 기술

FTP는 OSI 7 계층에서 무슨 계층인가?

정답 응용 계층

15 ③

- for i in range(1, 100): # i는 1부터 99까지 1씩 증가하며 반복한다.

i	sum += i	결과	cnt += 1
1	1	1	1
2	1+2	3	2
3	1+2+3	5	3
4	1+2+3+4	10	4
5	1+2+3+4+5	15	5
6	1+2+3+4+5+6	21	6

- i가 5가 되면 10이었던 sum은 5를 더해 15가 되고, cnt는 5가 된다. 15인 sum은 20 이상이 아니라 다시 반복문으로 올라간다.
- i가 6이 되고 15였던 sum은 6를 더해 21이 되고, cnt는 6이 된다. 21인 sum은 20 이상이기 때문에 참이 되어 break문을 만나 반복문을 벗어난다.
- print(cnt)를 수행하여 6이 출력된다.

16 ④

DDL(정의어) : CREATE(테이블 생성), DROP(테이블 삭제), ALTER(테이블 구조 변경)

오답 피하기

DELETE(튜플 삭제)은 DML(조작어)이다.

17 ①

- WHERE 컬럼 IS NULL: // NULL인 자료
- WHERE 컬럼 IS NOT NULL: // NULL이 아닌 자료

18 ②

- A 클래스 : 0(1,0,0,0 ~ 126,0,0,0)
- B 클래스 : 10(128,0,0,0 ~ 191,255,0,0)
- C 클래스 : 110(192,0,0,0 ~ 223,255,255,0)
- D 클래스 : 1110(멀티캐스트)
- E 클래스 : 1111(실현용)

19 ④

- 블랙박스 테스트 : 동등 분할, 경계값 분석, 오류 예측
- 화이트박스 테스트 : 기초경로 검사, 조건 검사, 루프 검사

20 ①

- 리눅스 파일 권한은 rwx = 4, 2, 1의 합으로 표현한다.

7			5			4		
4	2	1	4	2	1	4	2	1
R	W	X	R	-	X	R	-	-
소유자			그룹			기타		

- 소유자 : 읽기/쓰기/실행(7:모든 권한)
- 그룹 : 읽기/실행(5)
- 기타 : 읽기(4)
- 즉, 모든 권한은 70이다.

21 ④

교착상태 필요조건 4가지 : 상호배제, 점유와 대기, 순환 대기, 비선점

22 ②

구분	알파 테스트	베타 테스트
참여자	개발자, 내부 직원	일반 사용자
환경	개발 환경	실제 사용 환경
시점	개발 후반	출시 직전

오답 피하기

- 검증 : 개발자 관점(제대로 만들었는가?)
- 확인 : 사용자 관점(사용자 요구에 맞는가?)

23 ②

오답 피하기

- ① 트리 : 계층 구조 자료구조
- ③ 덱(Deque) : 양쪽 삽입 · 삭제 가능
- ④ 스택 : LIFO(후입선출) 자료구조

24 ①

- >(이상), <(초과), <=(이하), <=(미만)
- SELECT * FROM 학생 WHERE 나이 BETWEEN 23 AND 29;와 같다.

25 ②

range(10)는 0부터 9까지 1씩 증가하는 범위가 된다. 1씩 증가하는 i를 누적(더하기)하면 '0+1+2+...+8+9 = 45'이다.

26 ④

데이터베이스 설계 순서 : 요구 분석 → 개념적 설계 → 논리적 설계 → 물리적 설계 → 구현 → 운영 및 유지 보수

27 ④

데이터베이스 3단계 스키마

- 외부 스키마 : 사용자 관점, 뷰 정의
- 개념 스키마 : 전체 논리적 구조 정의
- 내부 스키마 : 물리적 저장 구조 정의

28 ①

오답 피하기

- ② cd : 디렉터리 이동
- ③ mkdir : 디렉터리 생성
- ④ rm : 파일, 디렉터리 삭제

29 ②

데이터베이스 관리 시스템(DBMS)

- 장점 : 중복 최소화, 일관성 유지, 무결성 보장, 보안 강화, 동시성 제어, 독립성 확보
- 단점 : 유지보수 비용 발생, 운영 및 관리 비용 증가, 복구가 어려움

30 ①

물리(비트), 데이터 링크(프레임), 네트워크(패킷), 전송(세그먼트), 세션/표현/응용(데이터)

31 ①

CI는 지속적 통합(Continuous Integration), CD는 지속적 배포(Continuous Deployment/ Delivery)를 의미한다.

32 ③

오답 피하기

- **[F4]+[D]** : 현재 창을 모두 최소화
- **[Alt]+[Tab]** : 창 전환
- **[F4]+[E]** : 파일 탐색기

+ 또 다른 기술

다음 중 윈도우에서 보안 모드(Safe Mode)로 진입하기 위한 단축키는?

정답 **[Shift]+재시작 또는 [F8]**

33 ④

[Shift]+[Delete]는 영구 삭제 단축키로, 선택한 파일이나 폴더를 휴지통으로 보내지 않고 즉시 하드 디스크에서 삭제한다.

34 ③

자바(Java)의 주석 처리 방식

- `//` : 한 줄 주석
- `/* */` : 여러 줄 주석
- `/** */` : 문서화 주석

오답 피하기

`<!-- -->`는 HTML의 주석 처리 방식이다.

35 ②

학습성 : 사용자가 처음 사용할 때 쉽게 배울 수 있는 정도

오답 피하기

- 직관성 : 설명이 없어도 바로 이해되는 정도
- 이식성 : 다른 환경에서도 사용 가능한 정도
- 효율성 : 숙련 사용자의 작업 속도

36 ④

`<img>`는 이미지 표시용 태그로, 문서의 구조와는 관계가 없다.

오답 피하기

- `<html>` : 문서 전체
- `<head>` : 문서 정보
- `<body>` : 화면에 보이는 내용

+ 또 다른 기술

다음 중 Html에서 태그 기호는 무엇인가?

- ① `< >`
- ② `{ }`
- ③ `[ ]`
- ④ `( )`

정답 ①

37 ③

CASCADE는 부모 테이블의 데이터가 삭제/변경되면, 이를 참조하고 있는 자식 테이블의 데이터도 자동으로 함께 처리된다.

38 ③

슈퍼키는 릴레이션 내의 튜플들을 식별할 수 있는 유일성은 만족하지만, 최소성은 만족하지 못한다.

오답 피하기

유일성과 최소성을 모두 만족하는 키

- 후보키 : 기본키가 될 수 있는 키
- 기본키 : 후보키 중 선택된 키
- 대체키 : 후보키 중 기본키로 선택되지 않은 키

39 ④

- 트랜잭션이 정상적으로 처리될 때의 상태 변화 순서 : active(실행) → partially committed(모든 연산 수행 완료, 아직 확정 전) → committed(영구 반영 완료)
- 트랜잭션이 실패했을 때의 상태 변화 순서 : active(트랜잭션 실행) → failed(오류 발생) → aborted(롤백 완료, 이전 상태로 복구)

+ 또 다른 기술

트랜잭션의 실행이 실패하여 수행된 연산이 모두 취소되어 원래 상태로 복구된 상태를 무엇이라고 하는가?

정답 철회(Aborted)

40 ②

Allocation(할당)은 메모리를 할당하는 과정을 의미한다.

오답 피하기

- Deallocation(해제) : 메모리 해제
- Compilation(컴파일) : 소스 코드를 기계어로 변환
- Initialization(초기화) : 초기값 설정

41 ①

Time Sharing(시분할 처리)은 CPU의 전체 사용 시간을 아주 짧은 시간 단위로 나누어 여러 사용자(또는 프로세스)에게 번갈아 할당하는 방식이다.

오답 피하기

- Batch Processing : 작업을 일정량을 모아 한꺼번에 처리(일괄 처리)
- Real Time Processing : 작업이 발생하자마자 바로 처리(즉시 처리)
- Multiprocessing : 여러 CPU를 사용하여 처리(다중 처리)

42 ④

도메인(Domain)은 하나의 속성(Attribute)이 가질 수 있는 원자값(Atomic Value)들의 집합 또는 범위를 의미한다.

오답 피하기

- ① 튜플들을 구분하는 범위 : 키(Key)의 역할
- ② 튜플들의 모음 : 릴레이션(Relation)
- ③ 속성들의 모음 : 릴레이션 스키마(Relation Schema)

+ 또 다른 기술

다음 중 하나의 속성이 취할 수 있는 같은 타입의 원자값들의 집합을 의미하는 것은?

정답 도메인

+ 또 다른 기술

릴레이션과 테이블의 차이점은?

정답 릴레이션(이론적)은 중복된 튜플을 허용하지 않고, 테이블(물리적)은 중복될 수 있다.

43 ④

UPDATE는 데이터를 갱신(수정)하는 DML(조작어)로써 보안이나 무결성 제어 목적은 아니다.

오답 피하기

- ① GRANT : 사용자에게 권한을 부여 → 보안 관련 명령어
- ② REVOKE : 사용자 권한 회수 → 보안 관련 명령어
- ③ COMMIT : 트랜잭션 완료, 데이터의 일관성 · 연속성 유지

44 ②

arr.length → 행 개수

오답 피하기

- arr[0].length → 첫 번째 행의 열 개수 → 결과 : 2
- arr[1].length → 두 번째 행의 열 개수 → 결과 : 3

45 ④

(개방적) public > protected > default > private (폐쇄적)

오답 피하기

- public : 어디서든 접근 가능
- protected : 같은 패키지 또는 상속 관계에서 접근 가능
- default : 같은 패키지 내에서 접근 가능
- private : 같은 클래스에서만 접근이 가능하며, 상속받은 자식 클래스에서도 접근 불가

46 ②

- cnt에는 0, i에는 10을 주고 while True를 만나 무한 반복을 한다.
- 조건문 if i % 2 == 0의 뜻은 "10과 2를 나눈 나머지 0이 i에 있는 0과 같니?"라는 뜻이고, 참이기 때문에 0인 cnt에 1을 더해 cnt는 1이 된다.
- 이 과정이 i가 20보다 크기 전인 21까지 반복하게 되면서 짝수의 개수를 cnt에 담게 된다.
- 결국 i가 짝수인 "10, 12, 14, 16, 18, 20"인 6이 cnt에 담기고 i가 21이 되면서 break를 만나 반복문을 벗어나게 되는 프로그램이다.

47 ④

- LAN 구조/방식(토큰 방식) : 토큰이라는 '권한'을 가진 노드만 데이터를 전송하므로 충돌이 발생하지 않는 확정적 방식(㉠ Token Ring, Token Bus, FDDI)
- LAN 접근 제어(경쟁 방식) : 특정 권한 없이 누구나 선로가 비어 있으면 데이터를 전송하며, 충돌을 감수하는 확률적 방식(예: CSMA/CD, CSMA/CA)

48 ③

file : 파일의 형식(타입)을 확인

오답 피하기

- ① grep : 파일 내에서 특정 문자열 검색, 결과 출력 가능
- ② more : 파일 내용을 페이지 단위로 출력
- ④ cat : 파일 내용을 한 번에 출력

49 ③

MODIFY : 기존 필드(열)의 자료형이나 길이 변경 시 사용

오답 피하기

- ① ADD : 새로운 필드 추가
- ② UPDATE : 데이터 값 수정(구조 변경 아님)
- ④ DROP : 필드 또는 테이블 삭제

50 ①

Flag는 데이터 통신에서 프레임의 시작과 끝을 구분하기 위해 사용하는 특수한 비트 패턴이다. 송신측은 프레임을 전송하기 직전에 11100111 패턴을 삽입하고, 데이터 전송이 완료되면 다시 11100111을 붙여 프레임의 경계를 표시한다.

51 ④

구분	TCP	UDP
연결 방식	연결형	비연결형
신뢰성	O	X
속도	느림	빠름
오류 제어	O	X

52 ②

클래스가 인터페이스를 사용할 때는 반드시 implements 키워드를 사용한다.

53 ②

소스 코드 분석

1. main을 먼저 실행한다.
2. A obj3 = new B(); // 참조 변수 타입 obj3, 실제 객체 타입 : B
3. obj3.greet(); // 실제 객체(B)의 메서드를 실행한다.
4. class B로 가서 B를 출력한다.

오답 피하기

class B extends A // B 클래스가 A 클래스를 상속함
@Override // 부모 클래스의 메서드를 재정의(오버라이딩)
→ 클래스 상속은 extends
→ 인터페이스 구현은 implements

54 ④

소스 코드 분석

1. A obj = new B(); // 참조 변수 타입 A, 실제 객체 타입 : B
2. obj.greet(); // 실제 객체인 B의 greet()를 실행한다.
3. super.greet(); // A클래스의 greet()를 실행, "A"를 출력하고 줄바꿈
4. System.out.println("B"); // B클래스로 돌아가 "B"를 출력하고 줄바꿈

55 ①

소산백업(Off-site Backup)은 백업한 데이터를 본사와 물리적으로 떨어진 원격지나 안전한 외부 장소에 보관하는 방식이다.

오답 피하기

- 부분백업 : 변경된 데이터 일부만 백업, 저장 공간은 절약되지만, 완전 복구는 어려움
- 전체백업 : 모든 데이터를 한 번에 백업하는 방식
- 증분백업 : 마지막 백업 이후 변경된 데이터만 백업, 복구 시 여러 백업이 필요

+ 또 다른 기술

백업 방식 중 마지막 풀 백업 이후 변경된 모든 데이터를 백업하는 방식은?

정답 차등백업

차등백업은 마지막 풀 백업 이후 변경된 모든 데이터를 매번 백업하는 방식으로 백업할수록 용량이 점점 커진다(복구 시 풀 백업 + 마지막 차등백업만 있으면 됨).

56 ④

POP3는 'Post Office Protocol version 3'의 약자로, 서버에 도착한 메일을 내 컴퓨터로 가져올 때 사용하는 수신용 프로토콜이다. 기본 포트번호는 110을 사용한다.

오답 피하기

시험에 많이 나오는 포트 번호

- FTP : 21
- SMTP : 25
- DNS : 53
- IMAP : 143
- HTTP : 80
- HTTPS : 443

57 ③

오답 피하기

- ① 라우팅 : RIP, OSPF
- ② 파일 전송 : FTP
- ④ 웹 통신 : HTTP

58 ①

- 블록 크기(두 번째 옥텟을 이용하여 계산) : $256 - 240 = 16$
- 두 번째 옥텟 값은 16의 배수(0, 16, 32, 48, 64, 80, 96, 112, ...)가 올 수 있다.

59 ③

STP(Spanning Tree Protocol)는 2계층(Data Link Layer) 스위치 환경에서 루프(Loop) 현상을 방지하기 위해 사용하는 프로토콜이다.

오답 피하기

- ① OSPF : 링크 상태 기반, 대규모 네트워크에서 사용, 네트워크 계층
- ② BGP : 자율 시스템(AS) 간 경로 설정, 인터넷 핵심 프로토콜, 네트워크 계층
- ④ RIP : 거리 벡터 방식, 홉 수 기준, 네트워크 계층

60 ②

오답 피하기

- ① UNION ALL : 중복을 제거하지 않고 모두 합쳐서 검색
- ③ UNIQUE : 테이블 생성문에서 데이터 입력의 중복을 방지하는 명령
- ④ DISTINCT : SELECT문 내부에서 사용하는 중복을 제거하는 명령

또, 드릴게요! 이기적이 준비한 선물

또기적 합격자료집

〈비매품〉



또기적 합격자료집

저작권자의 허락 없이 도서의 내용을 그대로 복사하여 다른 출판물이나 온라인에 게재하는 것은 저작권법 위반에 해당합니다.